

```
In [ ]: from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
In [ ]: import pandas as pd
pd.read_csv("/content/Dataset 2.csv")
```

Out [5]:

	UserID	Age	Gender	SubscriptionType	WatchHoursPerWeek	DevicesUsed	FavoriteGenre	AdClicks	MonthlySpend	SubscriptionRenewed
0	1001	22	Female	Basic	23	5	Comedy	13	353	No
1	1002	55	Male	Basic	9	4	Drama	14	317	Yes
2	1003	49	Male	Basic	8	3	Comedy	16	309	No
3	1004	39	Female	Premium	19	5	Drama	45	833	Yes
4	1005	38	Female	Premium	23	5	Sci-Fi	24	804	Yes
...	...	...	...	...	...	...	...	...	...	...
745	1746	35	Male	Premium	13	5	Action	47	732	No
746	1747	43	Female	VIP	11	3	Comedy	33	1388	No
747	1748	33	Female	Premium	19	4	Action	28	817	Yes
748	1749	33	Male	Basic	20	4	Horror	26	451	Yes
749	1750	25	Female	Premium	12	3	Horror	10	802	No

750 rows × 10 columns

```
In [ ]: df=pd.read_csv("/content/Dataset 2.csv")

df.head()
```

Out [7]:

	UserID	Age	Gender	SubscriptionType	WatchHoursPerWeek	DevicesUsed	FavoriteGenre	AdClicks	MonthlySpend	SubscriptionRenewed
0	1001	22	Female	Basic	23	5	Comedy	13	353	No
1	1002	55	Male	Basic	9	4	Drama	14	317	Yes
2	1003	49	Male	Basic	8	3	Comedy	16	309	No
3	1004	39	Female	Premium	19	5	Drama	45	833	Yes
4	1005	38	Female	Premium	23	5	Sci-Fi	24	804	Yes

```
In [ ]: df.shape
```

Out [9]: (750, 10)

```
In [ ]: df.columns
```

Out [10]: Index(['UserID', 'Age', 'Gender', 'SubscriptionType', 'WatchHoursPerWeek', 'DevicesUsed', 'FavoriteGenre', 'AdClicks', 'MonthlySpend', 'SubscriptionRenewed'], dtype='object')

```
In [ ]: df.isnull().sum()
```

Out [11]:

	0
UserID	0
Age	0
Gender	0
SubscriptionType	0
WatchHoursPerWeek	0
DevicesUsed	0
FavoriteGenre	0
AdClicks	0
MonthlySpend	0
SubscriptionRenewed	0

dtype: int64

```
In [ ]: df['Age'].mean()
```

```
Out [14]: np.float64(41.824)
```

```
In [ ]: df['WatchHoursPerWeek'].mean()  
df['MonthlySpend'].mean()
```

```
Out [19]: np.float64(689.9053333333334)
```

```
In [ ]:   
  
renewal_pct=(df['SubscriptionRenewed'].value_counts(normalize=True) * 100)  
print(renewal_pct)
```

```
SubscriptionRenewed  
No      53.733333  
Yes     46.266667  
Name: proportion, dtype: float64
```

```
In [ ]: import pandas as pd  
df=pd.read_csv("/content/Dataset 2.csv")  
df_encoded = pd.get_dummies(df, columns=['Gender', 'SubscriptionType', 'FavoriteGenre'], drop_first=True)
```

```
In [ ]: X = df_encoded.drop(columns=['UserID', 'SubscriptionRenewed'])  
y = df_encoded['SubscriptionRenewed'].map({'Yes': 1, 'No': 0})
```

```
In [ ]:   
  
from sklearn.model_selection import train_test_split  
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
In [ ]:   
  
from sklearn.tree import DecisionTreeClassifier  
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report  
dt_model = DecisionTreeClassifier(random_state=42)  
dt_model.fit(X_train, y_train)  
dt_preds = dt_model.predict(X_test)  
dt_acc = accuracy_score(y_test, dt_preds)  
print(f"Decision Tree Accuracy: {dt_acc:.2f}")  
print("Confusion Matrix:\n", confusion_matrix(y_test, dt_preds))
```

```
Decision Tree Accuracy: 0.53  
Confusion Matrix:  
[[43 39]  
 [32 36]]
```

```
In [ ]:   
  
from sklearn.neighbors import KNeighborsClassifier  
knn_model = KNeighborsClassifier(n_neighbors=5)  
knn_model.fit(X_train, y_train)  
knn_preds = knn_model.predict(X_test)  
knn_acc = accuracy_score(y_test, knn_preds)  
print(f"Decision Tree Accuracy: {dt_acc:.4f}")  
print(f"KNN Accuracy:          {knn_acc:.4f}")
```

```
Decision Tree Accuracy: 0.5267  
KNN Accuracy:          0.6333
```

```
In [35]:   
  
from sklearn.linear_model import LinearRegression  
df_encoded=pd.get_dummies(df)  
# Q19: Set new target for regression  
X_reg = df_encoded.drop(columns=['UserID', 'MonthlySpend'])  
y_reg = df_encoded['MonthlySpend']  
  
X_train_r, X_test_r, y_train_r, y_test_r = train_test_split(X_reg, y_reg, test_size=0.2, random_state=42)  
  
lr_model = LinearRegression()  
lr_model.fit(X_train_r, y_train_r)  
  
# Q20: Predict for a fake new user (must have same number of columns/features as X_reg)  
import numpy as np  
sample_user = np.array([X_reg.iloc[0]]) # Using the first user's shape configuration as an example  
predicted_spend = lr_model.predict(sample_user)  
print(f"Predicted Monthly Spend for sample user: ${predicted_spend[0]:.2f}")
```

```
Predicted Monthly Spend for sample user: $414.11
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but  
LinearRegression was fitted with feature names
```

